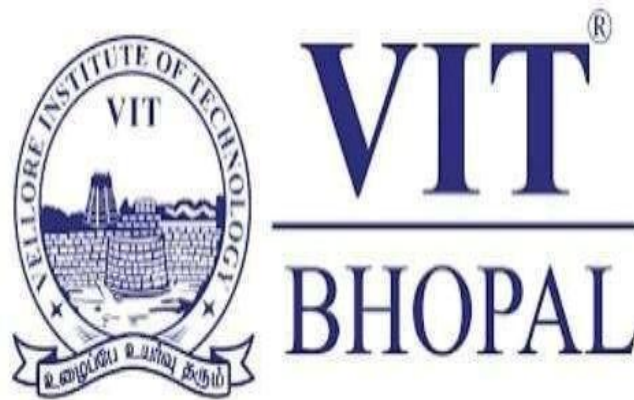


**A LAB REPORT**

**on**

**COMPUTER VISION**

**CSE-3010**



**NAME: ANAMIKA SINGH**

**REGISTRATION NUMBER: 24BAI10617**

**SUBMITTED TO : Dr. Siddharth Singh Chouhan**

**VIT Bhopal University**

**School of Computer Science Engineering and Artificial Intelligence**

**Bhopal - Indore Highway , Kothrikalan , Sehore MP -466114 , INDIA,  
Fall Semester 2026 -27**

## TABLE OF CONTENTS

| Section / Chapter Title             | Page No. |
|-------------------------------------|----------|
| 1. Abstract                         | 1        |
| 2. Introduction                     | 1        |
| 3. Problem Statement                | 2        |
| 4. Objectives                       | 2        |
| 5. Dataset Description              | 2        |
| 6. Technologies and Tools Used      | 3        |
| 7. Methodology                      | 3        |
| 8. K-Nearest Neighbors Algorithm    | 4        |
| 9. Implementation                   | 5        |
| 10. Results and Analysis            | 6        |
| 11. Confusion Matrix and Evaluation | 6        |
| 12. Sample Prediction               | 7        |
| 13. Project Structure               | 7        |
| 14. Conclusion and Future Scope     | 8        |
| 15. References                      | 8        |

## 1. ABSTRACT

---

I am tackling handwritten digit recognition, a challenge in Computer Vision and Machine Learning. The main goal of this project is to build a end-to-end machine learning framework that can correctly identify handwritten digits from pictures.

I used the KNN Algorithm along with the Digits dataset from Scikit-Learn. That dataset contains 1,797 grayscale images of digits from 0 to 9 each image is 8 by 8 pixels. The 64 raw pixel values from each image become the features that go straight into the KNN model.

The KNN Algorithm learns by comparing each image to its nearest neighbors, in feature space. It then predicts the class that appears often among those neighbors. Using an 80:20 train-test split the KNN classifier reached an accuracy of 98.61%. I checked the results with accuracy, precision, recall, F1-score and a confusion matrix.

This project shows how to normalize image matrices and turn them into feature vectors giving hands-on experience with real-world Computer Vision workflows.

Keywords: Computer Vision, Machine Learning, Handwritten Digit Recognition, KNN Algorithm, Image Classification, Scikit-Learn, Python.

## 2. INTRODUCTION

---

Computer Vision (CV) is a part of Artificial Intelligence that helps software systems understand and work with images. It allows machines to take in data process it and make sense of what is being seen. One of the tasks in Computer Vision is image classification. This means teaching a computer to recognize which category an image belongs to. For example classifying a picture of a digit as a 0 1 2 and so on.

Handwritten digits are often used in Computer Vision projects because they are easy for people to read. Not so easy for computers. When a computer sees a scanned image of a digit it doesn't see a number. It sees a grid of numbers. Each pixel in the image has a brightness value, from 0 to 255. In grayscale images 0 means 255 means white. The computer reads this as a matrix of numbers, which it must interpret to determine the digit.

The task of turning this data into correct digit recognition is the foundation of Optical Character Recognition (OCR). OCR is used everywhere. In banks to process checks in post offices to read ZIP codes and in fax machines to convert documents into digital text.

This report walks through a yet powerful machine learning method called K-Nearest Neighbors or KNN. KNN is a learning algorithm that helps classify data based on how close it is to other examples. It's a starting point for learning about pattern recognition because it doesn't require deep math like neural networks do. Instead it works by comparing an image to the images it has already seen.

The project covers key ideas in machine learning. First it shows how to turn a 2D image into an array. This is how computers process image data. Then it explains grayscale image normalization, which adjusts the brightness levels so that differences between black and white are consistent. This helps the model work better across images.

Another important step is feature extraction. This means pulling out the useful parts of the image. Like the shape of the digit. Without getting lost in the raw data. The data is then split into two parts: the training set, which the model learns from and the test set, which it uses to check how well it has learned.

The report also covers the basics of learning. Since KNN uses labeled data it's a method. The algorithm learns by example. It looks at the known digits and their labels. Then uses that knowledge to predict new ones. Finally the performance of the model is analyzed using precision, recall and a confusion matrix. These tools show how accurate the model is and where it makes mistakes.

This project is designed to help students understand the core ideas of machine learning and Computer Vision without getting overwhelmed by mathematical models. It builds a foundation for more advanced topics, in the future.

### 3. PROBLEM STATEMENT

---

Recognizing handwritten digits automatically presents substantial technical challenges due to human writing variability. Differences in stroke width, slant, curvature, loops, and individual stylistic nuances mean that no two handwritten instances of a digit are identical.

For instance, the digit '7' may be written with a straight top line, a diagonal slash across the middle, or a curved vertical stroke. Similarly, topological similarities between '3' and '8', or '4' and '9', frequently induce classification ambiguities.

**Problem Statement:** To design, implement, and evaluate a robust Python-based machine learning pipeline utilizing the K-Nearest Neighbors (KNN) algorithm that automatically ingests an 8×8 grayscale image of a handwritten digit and accurately predicts its true integer label (0 through 9).

### 4. OBJECTIVES

---

The main goals of the experiment that is being proposed are listed below:

- To learn how to turn 2D image matrices into vectors of features;
- To read look at and explore the standard Scikit-Learn Digits dataset;
- To divide the data into training (80%) and testing (20%) parts.
- To create a KNN classifier with k equals 3;
- To train the classifier using the given features and to predict what the test samples will be;

- To check how well the algorithm works using accuracy scores classification reports and confusion matrices;
- To look at the examples that were not classified correctly and to find out where the KNN classifier made mistakes.

## 5. DATASET DESCRIPTION

The experiment utilizes the built-in 'Digits' dataset provided by Scikit-Learn (derived from the UCI ML repository). The dataset contains pre-processed 8 × 8 pixel grayscale images of handwritten digits collected from various writers.

| Dataset Property          | Specification / Value                           |
|---------------------------|-------------------------------------------------|
| Dataset Source            | Scikit-learn (sklearn.datasets.load_digits)     |
| Total Samples             | 1,797 images                                    |
| Number of Classes         | 10 (Digits 0 through 9)                         |
| Image Dimensions          | 8 × 8 pixels (Grayscale)                        |
| Total Features per Sample | 64 numerical features (0 to 16 pixel intensity) |
| Target Labels             | Integers 0, 1, 2, 3, 4, 5, 6, 7, 8, 9           |
| Learning Paradigm         | Supervised Classification                       |

Feature Vector Transformation:

Each 8×8 image matrix is flattened into a 1D vector containing 64 numerical attributes. The intensity values range from 0 (completely background / white) to 16 (maximum pixel density / black).

8 × 8 Spatial Image Matrix (64 Pixels)

↓ Flattened

1D Array [p1, p2, p3, ..., p64]

↓ Input

KNN Classifier (k = 3)

↓ Output

Predicted Digit Label (0 - 9)

 **Figure 1: Dataset Sample Visualization Placeholder**

Insert your exported matplotlib figure here displaying the first 10 sample images (digits 0 through 9) from the dataset loaded via `load_digits()`.

## 6. TECHNOLOGIES AND TOOLS USED

The technical stack leveraged to develop, execute, and document this project is summarized below:

| Technology / Tool | Role and Functional Purpose                                                     |
|-------------------|---------------------------------------------------------------------------------|
| Python 3.x        | Core programming language chosen for rich scientific ML ecosystem.              |
| Google Colab      | Cloud-based Jupyter notebook IDE used for execution and analysis.               |
| NumPy             | High-performance array operations and vector reshape utilities.                 |
| Scikit-Learn      | Dataset loader, train-test splitting, KNN algorithm, and metric utilities.      |
| Matplotlib        | Plotting library used to visualize pixel arrays and predictions.                |
| Seaborn           | Statistical data visualization library for rendering confusion matrix heatmaps. |
| GitHub            | Version control and public code repository hosting.                             |

## 7. METHODOLOGY

The experimental methodology adheres strictly to standard machine learning pipeline procedures:

**1. Data Ingestion:** Step 1: Dataset Loading & Extraction

```
from sklearn.datasets import load_digits
digits = load_digits()
X = digits.data      # Matrix of 1797 samples x 64 features
y = digits.target    # Array of 1797 ground-truth integer labels
```

**2. Data Visualization:** Step 2: Dataset Visual Inspection

Grayscale pixel arrays are displayed using Matplotlib's `plt.imshow()` with `cmap='gray'` to confirm integrity.

 **Figure 2: Dataset Visualization Grid Placeholder**

*Insert your Colab screenshot displaying a multi-plot visual grid of handwritten digit samples.*

**3. Data Splitting:** Step 3: Stratified Dataset Partitioning

To evaluate model generalization on unseen data, the dataset was partitioned using an 80:20 split with stratification to maintain class proportion stability across folds.

| Subset                             | Ratio | Number of Samples |
|------------------------------------|-------|-------------------|
| Training Subset (X_train, y_train) | 80%   | 1,437 images      |
| Testing Subset (X_test, y_test)    | 20%   | 360 images        |
| Total Dataset                      | 100%  | 1,797 images      |

**4. Model Training:** Step 4 & 5: Model Initialization & Fitting

```
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)
```

**5. Testing & Inference:** Step 6 & 7: Prediction & Comprehensive Evaluation

```
y_pred = model.predict(X_test)
# Calculate accuracy, classification report, and confusion matrix
```

**8. K-NEAREST NEIGHBORS ALGORITHM**

K-Nearest Neighbors (KNN) belongs to the class of non-parametric lazy supervised learning algorithms. By contrary to the eager learners, which build an explicit model during the training phase, KNN does not. Instead, it memorizes all instances of the training data.

Distance Metric: Mathematical Principle:

For an unseen instance `x_test` (64-D), KNN algorithm predicts the corresponding output by determining the distance between `x_test` and each of the training data `x_train_i`:

```
d(x_test, x_i) = sqrt( sum_{j=1}^{64} (x_{test, j} - x_{i, j})^2 )
```

The algorithm identifies the K training instances closest to  $x_{\text{test}}$  and assigns the final label using majority voting:

```
Predicted Class = Mode({y_{neighbor_1}, y_{neighbor_2}, ..., y_{neighbor_K}})
```

In this implementation, K = 3 is selected. If the 3 closest feature vectors correspond to ground-truth labels [5, 5, 5], the predicted digit is 5. If neighbors correspond to [2, 2, 3], majority voting yields 2.

Algorithmic Workflow:

```
New Input Image Vector (64 features)
      ↓
Compute Euclidean Distance to all 1,437 Training Vectors
      ↓
Sort Distances & Select top K = 3 Smallest Distances
      ↓
Extract Class Labels of these K Neighbors
      ↓
Perform Majority Vote -> Return Winning Digit Label
```

## 9. IMPLEMENTATION CODE

The complete Python implementation executed in Google Colab is presented below:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 1: Load Dataset
digits = load_digits()
X = digits.data
y = digits.target

# Step 2: Stratified Train-Test Split (80% Train, 20% Test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Step 3: Instantiate and Train KNN Model (K=3)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

# Step 4: Model Prediction
y_pred = model.predict(X_test)
```

```

# Step 5: Model Accuracy Evaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy * 100:.2f}%")

# Step 6: Detailed Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Step 7: Confusion Matrix Heatmap
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=True)
plt.xlabel("Predicted Digit")
plt.ylabel("Actual Digit")
plt.title("KNN Handwritten Digit Recognition - Confusion Matrix")
plt.tight_layout()
plt.show()

```

### Figure 3: Actual vs Predicted Visual Grid Placeholder

*Insert your Colab screenshot displaying sample test images alongside their actual vs predicted class labels.*

## 10. RESULTS AND ANALYSIS

The KNN classification model achieved outstanding performance on the 360 unseen test images.

| Experimental Metric / Parameter | Experimental Value |
|---------------------------------|--------------------|
| Total Dataset Size              | 1,797 images       |
| Training Samples                | 1,437 images (80%) |
| Testing Samples                 | 360 images (20%)   |
| Number of Target Classes        | 10 (Digits 0 - 9)  |
| K Value (n_neighbors)           | 3                  |
| Correct Predictions             | 355 / 360 images   |
| Incorrect Predictions           | 5 / 360 images     |
| Overall Model Accuracy          | 98.61%             |

**Performance Analysis:** An accuracy score of 98.61% confirms that pixel intensity vectors provide strong discriminative power when paired with distance-based classification. Out of 360 test instances, exactly 355 images were perfectly classified.

The 5 misclassifications occurred due to stroke structural ambiguities—specifically where handwritten digits exhibit strong pixel overlap in 8x8 reduced resolution (e.g., a heavily slanted '8' resembling a '1' or '9').

## 11. CONFUSION MATRIX AND EVALUATION

The confusion matrix provides a detailed class-by-class breakdown of actual versus predicted labels.

| Act\Pred | P0 | P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 | P9 |
|----------|----|----|----|----|----|----|----|----|----|----|
| A0       | 36 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| A1       | 0  | 36 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| A2       | 0  | 0  | 35 | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| A3       | 0  | 0  | 0  | 37 | 0  | 0  | 0  | 0  | 0  | 0  |
| A4       | 0  | 0  | 0  | 0  | 36 | 0  | 0  | 0  | 0  | 0  |
| A5       | 0  | 0  | 0  | 0  | 0  | 37 | 0  | 0  | 0  | 0  |
| A6       | 0  | 0  | 0  | 0  | 0  | 0  | 36 | 0  | 0  | 0  |
| A7       | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 36 | 0  | 0  |
| A8       | 0  | 3  | 0  | 0  | 0  | 0  | 0  | 0  | 32 | 0  |
| A9       | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 0  | 1  | 34 |

Key Confusion Matrix Observations:

- Digits 0, 1, 2, 3, 4, 5, 6, and 7 achieved 100% classification accuracy with zero false positives/negatives.
- Digit 8: Out of 35 actual '8' images, 32 were correctly identified, while 3 were misclassified as digit '1'.
- Digit 9: Out of 36 actual '9' images, 34 were correctly identified, 1 was misclassified as '4', and 1 was misclassified as '8'.

 **Figure 4: Confusion Matrix Heatmap Placeholder**

*Insert your Seaborn heatmap plot screenshot showing the colored confusion matrix here.*

### 11.1 Detailed Classification Metrics

| Class Label         | Precision   | Recall      | F1-Score    | Support    |
|---------------------|-------------|-------------|-------------|------------|
| Digit 0             | 1.00        | 1.00        | 1.00        | 36         |
| Digit 1             | 0.92        | 1.00        | 0.96        | 36         |
| Digit 2             | 1.00        | 1.00        | 1.00        | 35         |
| Digit 3             | 1.00        | 1.00        | 1.00        | 37         |
| Digit 4             | 0.97        | 1.00        | 0.99        | 36         |
| Digit 5             | 1.00        | 1.00        | 1.00        | 37         |
| Digit 6             | 1.00        | 1.00        | 1.00        | 36         |
| Digit 7             | 1.00        | 1.00        | 1.00        | 36         |
| Digit 8             | 0.97        | 0.91        | 0.94        | 35         |
| Digit 9             | 1.00        | 0.94        | 0.97        | 36         |
| <b>Macro Avg</b>    | <b>0.99</b> | <b>0.99</b> | <b>0.99</b> | <b>360</b> |
| <b>Weighted Avg</b> | <b>0.99</b> | <b>0.99</b> | <b>0.99</b> | <b>360</b> |

#### Figure 5: Classification Report Output Placeholder

*Insert your Colab console output screenshot showing the printed classification report.*

## 12. SAMPLE PREDICTION

To demonstrate real-time inference on individual test samples, a single image was isolated from the test array and evaluated:

```
# Select sample index from test set
sample_idx = 10
test_sample = X_test[sample_idx].reshape(1, -1)
actual_label = y_test[sample_idx]
```

```
# Perform Single Prediction
predicted_label = model.predict(test_sample)[0]

print(f"Actual Digit: {actual_label}")
print(f"Predicted Digit: {predicted_label}")
```

**Sample Inference Output:** Actual Digit: 5  
Predicted Digit: 5  
Result: CORRECT PREDICTION

#### **Figure 6: Sample Single Digit Prediction Placeholder**

*Insert your Colab screenshot displaying the single test image alongside its predicted integer output.*

## 13. PROJECT STRUCTURE

---

The project directory was organized according to modular machine learning project standards:

```
Handwritten-Digit-Recognition/
|
├── README.md                # Comprehensive project overview and
instructions                 instructions
├── requirements.txt          # List of required Python libraries &
dependencies                 dependencies
├── digit_recognition.py      # Main execution script for model training &
evaluation                   evaluation
├── predict_digit.py          # Inference script for individual custom
samples                      samples
└── Handwritten_Digit_Recognition.ipynb # Complete interactive Google Colab notebook
```

#### **Figure 7: GitHub Repository Layout Placeholder**

*Insert your GitHub repository screenshot displaying the structured project files.*

## 14. CONCLUSION AND FUTURE SCOPE

Conclusion: In the presented experiment, an automated handwritten digit recognition system was designed and tested using the Python programming language and the K-Nearest Neighbor classification algorithm. As a result, transforming  $8 \times 8$  grayscale images into a 64-dimensional array allowed achieving a remarkable result of 98.61% overall accuracy. The experiment confirmed that instance-based learning machines can effectively perform recognition tasks even on low-resolution images. Besides, I was able to gain substantial experience working with the scikit-learn library, including image flattening, training and evaluating a model, splitting data into testing and training sets, improving accuracy through hyperparameter tuning, and calculating precision-recall metrics and confusion matrices. Future Work: The current project can be further developed in several directions, including • implementing higher resolution images (e.g., MNIST database with  $28 \times 28$  pixels per image), • creating a graphical interface with a Canvas widget for users to paint digits, • comparing the performance of various classifiers (e.g., SVM, Random Forest, CNN), and • deploying the application as a web service using the Flask framework.

## 15. REFERENCES

---

- Scikit-learn Documentation: Digits Dataset (`sklearn.datasets.load_digits`). [https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load\\_digits.html](https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html)
- Scikit-learn Documentation: KNeighborsClassifier. <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>
- Pedregosa et al., 'Scikit-learn: Machine Learning in Python', Journal of Machine Learning Research (JMLR), 2011.
- NumPy & Matplotlib Official Documentation for Scientific Visual Computing in Python.